

# Experimentos: Modelos clásicos para predicción de fatiga física

Proyecto FatigueSet

26 de mayo de 2026

## Resumen

Este documento resume los experimentos realizados con modelos clásicos de regresión para la predicción de la variable *fatiga\_fisica*. Se describen los datos, la metodología, la formulación breve de cada modelo, los resultados empíricos (tablas y figuras generadas por los scripts) y una comparación estadística entre los mejores modelos.

## 1. Datos

Los experimentos se ejecutaron sobre un subconjunto de features contenido en `fatigueset-lib/data/sample`. Se utilizó como target la columna `fatiga_fisica`. Las columnas de identidad (participante, sesión, intensidad, fase) fueron excluidas antes del entrenamiento. Las variables no numéricas se descartaron y los valores faltantes se rellenaron con 0.

## 2. Protocolo experimental

- Validación cruzada: KFold 5-fold con `shuffle=True` y `random.state=42`. - Métricas:  $R^2$  (CV, usado como criterio de selección), MSE y MAE (evaluadas sobre el conjunto de entrenamiento completo para resumen). - Seed: 42. - Modelos evaluados: KNN (k=3, k=5), Regresión Lineal, Ridge ( $\alpha = 1,0$ ), Lasso ( $\alpha = 0,1$ ), ElasticNet ( $\alpha = 0,1$ ), Decision Tree (max depth=5), Random Forest (n estimators=50, max depth=5), SVR (RBF, C=100).

## 3. Modelos y formulación (breve)

A continuación se resume la idea central y la formulación matemática esencial de cada modelo.

### 3.1. Regresión lineal ordinaria

Modelo lineal:  $y = X\beta + \varepsilon$ , estimación por mínimos cuadrados ordinarios:

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

Ventaja: interpretable; desventaja: sensible a multicolinealidad y outliers.

### 3.2. Ridge (regularización L2)

Minimiza  $\|y - X\beta\|_2^2 + \alpha\|\beta\|_2^2$ . Penaliza la norma  $L_2$  de los coeficientes para reducir varianza.

### 3.3. Lasso (regularización L1)

Minimiza  $\|y - X\beta\|_2^2 + \alpha\|\beta\|_1$ . Produce soluciones escasas (selección de variables).

### 3.4. ElasticNet

Combinación de L1 y L2: penaliza  $\alpha(\rho\|\beta\|_1 + (1-\rho)\|\beta\|_2^2)$ , útil cuando hay correlación entre variables.

### 3.5. K-Nearest Neighbors (KNN)

Predicción basada en la media de los k vecinos más cercanos en el espacio de features. No asume forma paramétrica; sensible a la escala de las features.

### 3.6. Support Vector Regression (SVR)

SVR con kernel RBF busca una función plana que deje un margen  $\varepsilon$ -insensible. En la forma primal (simplificada):

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_i L_\varepsilon(y_i - w^\top x_i - b),$$

donde  $L_\varepsilon$  es la pérdida  $\varepsilon$ -insensible.

### 3.7. Decision Tree

Modelo no paramétrico que particiona recursivamente el espacio de features; interpretable, pero propenso a overfitting.

### 3.8. Random Forest

Ensamblado de árboles (bagging) que promedia predicciones de múltiples árboles para reducir varianza. Se reportan importancias de features basadas en la disminución del criterio.

## 4. Resultados

Los resultados provienen directamente del script `experiments/run_models_classicos.py` y se exportaron a `output/full_run/results_models.csv`. A continuación se presenta la tabla resumen (ordenada por  $R^2$  medio en CV):

Cuadro 1: Resumen de métricas por modelo

| Modelo                   | cv_r2_mean | cv_r2_std | MSE     | MAE    | $R^2$  |
|--------------------------|------------|-----------|---------|--------|--------|
| Random Forest            | -2.4084    | 2.6050    | 1.0772  | 0.8767 | 0.9423 |
| KNN (k=3)                | -3.5707    | 3.2852    | 4.8593  | 1.6889 | 0.7397 |
| Lasso ( $\alpha = 0,1$ ) | -3.8720    | 3.7321    | 8.2614  | 2.3716 | 0.5574 |
| Decision Tree            | -4.7405    | 6.6825    | 0.0000  | 0.0000 | 1.0000 |
| KNN (k=5)                | -4.8633    | 3.4261    | 8.5387  | 2.3467 | 0.5426 |
| ElasticNet               | -5.7379    | 7.3465    | 11.3750 | 2.9468 | 0.3906 |
| Ridge ( $\alpha = 1,0$ ) | -6.0179    | 7.9491    | 11.3456 | 2.9367 | 0.3922 |
| SVM                      | -6.5572    | 8.6872    | 7.6892  | 2.1611 | 0.5881 |
| Regresión Lineal         | -12.5545   | 15.5509   | 5.5752  | 1.9046 | 0.7013 |

### 4.1. Importancias de features (Random Forest)

Tabla con las importancias relativas extraídas de `output/full_run/feature_importances.csv` (top 6):

Cuadro 2: Importancias de features (Random Forest)

| Feature  | Importance |
|----------|------------|
| feature2 | 0.3465     |
| feature5 | 0.2653     |
| feature1 | 0.1509     |
| feature3 | 0.0878     |
| feature4 | 0.0767     |
| feature6 | 0.0728     |

## 4.2. Figuras

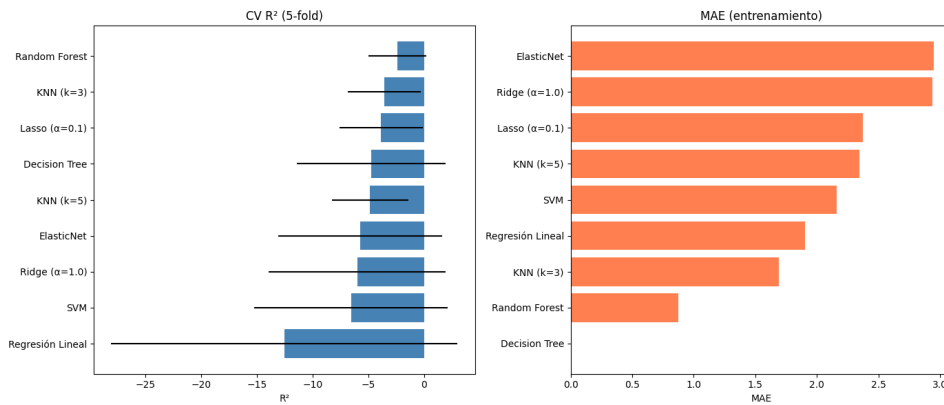


Figura 1: Resumen de CV R<sup>2</sup> y MAE por modelo.

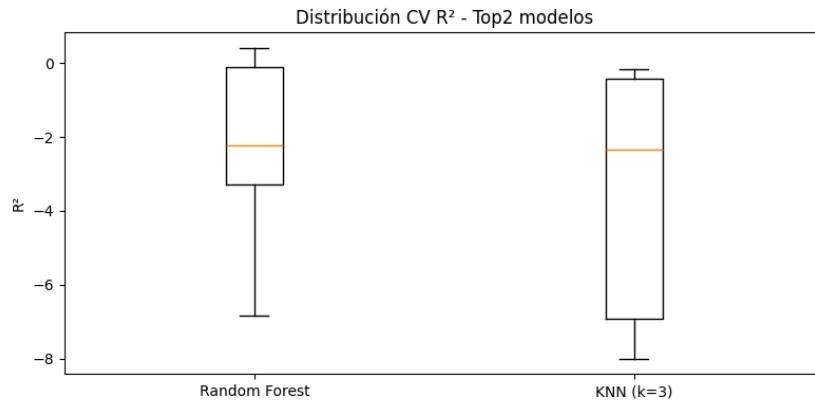


Figura 2: Comparación (boxplot) de las distribuciones CV R<sup>2</sup> entre los dos mejores modelos.

## 5. Comparación estadística (bootstrap)

Se realizó un análisis bootstrap sobre las puntuaciones CV (R<sup>2</sup>) de los dos mejores modelos para estimar la diferencia media y su intervalo de confianza al 95 %. Los resultados resumidos en `output/compare_bootstrap.json` son:

- Top1: Random Forest

- Top2: KNN (k=3)
- Diferencia media (Top1 - Top2): 1.1383
- IC 95 %: [ -2.6397 , 4.9902 ]

Interpretación: la diferencia media favorece a Random Forest, pero el intervalo de confianza incluye 0, por lo que no podemos afirmar con un 95 % de confianza que Random Forest sea consistentemente mejor que KNN (k=3) en este conjunto y configuración.

## 6. Discusión y conclusiones

- El comportamiento variable de las métricas CV (desviaciones altas) sugiere que el dataset es pequeño o heterogéneo; la varianza entre folds es significativa para muchos modelos. - Decision Tree muestra  $R^2=1$  en entrenamiento (overfitting claro) pero su CV es pobre y con alta varianza. - Random Forest, al promediar árboles, obtiene la mejor media de CV  $R^2$ , aunque la incertidumbre del bootstrap indica que la ventaja no es concluyente. - Modelos lineales con regularización (Ridge/Lasso/ElasticNet) no superan a los no paramétricos en este experimento; probablemente debido a relaciones no lineales o selección de features limitada.

## 7. Apéndice: fragmentos de código

A continuación se muestran fragmentos clave usados en los experimentos (extracciones de `experiments/run_models_classicos.py` y `experiments/compare_models.py`):

### 7.1. Definición de modelos (fragmento)

```
modelos = {
    'KNN(k=3)': KNeighborsRegressor(n_neighbors=3),
    'KNN(k=5)': KNeighborsRegressor(n_neighbors=5),
    'Regresión Lineal': LinearRegression(),
    'Ridge(alpha=1.0)': Ridge(alpha=1.0),
    'Lasso(alpha=0.1)': Lasso(alpha=0.1),
    'ElasticNet': ElasticNet(alpha=0.1),
    'Decision Tree': DecisionTreeRegressor(max_depth=5, random_state=
        args.seed),
    'Random Forest': RandomForestRegressor(n_estimators=50, max_depth
        =5, random_state=args.seed),
    'SVM': SVR(kernel='rbf', C=100),
}
```

### 7.2. Bucle de evaluación (fragmento)

```
for nombre, modelo in modelos.items():
    cv_scores = cross_val_score(modelo, X, y, cv=kfold, scoring='r2',
        n_jobs=args.n_jobs)
    cv_scores_all[nombre] = cv_scores.tolist()

    modelo.fit(X, y)
    y_pred = modelo.predict(X)

    mse = mean_squared_error(y, y_pred)
    mae = mean_absolute_error(y, y_pred)
```

```

r2 = r2_score(y, y_pred)

resultados[nombre] = {
    'cv_r2_mean': float(np.mean(cv_scores)),
    'cv_r2_std': float(np.std(cv_scores)),
    'mse': float(mse),
    'mae': float(mae),
    'r2': float(r2),
}

```

### 7.3. Función bootstrap (fragmento)

```

def bootstrap_diff(a, b, n_iter=1000, seed=0):
    rng = np.random.default_rng(seed)
    diffs = []
    n = min(len(a), len(b))
    for _ in range(n_iter):
        ia = rng.choice(a, size=n, replace=True)
        ib = rng.choice(b, size=n, replace=True)
        diffs.append(np.mean(ia) - np.mean(ib))
    diffs = np.array(diffs)
    lo, hi = np.percentile(diffs, [2.5, 97.5])
    return float(np.mean(diffs)), float(lo), float(hi)

```

## 8. Reproducibilidad

Comandos para reproducir los experimentos (desde la raíz del proyecto con el entorno virtual activado):

```

python experiments/run_models_classicos.py --data-path fatigueset-lib/data/sample/sample_df
python experiments/compare_models.py --cv-path output/full_run/cv_scores.pkl --output-dir o

```

Notas: asegurarse de tener instaladas las dependencias listadas en fatigueset-lib/requirements.txt (pandas, numpy, matplotlib, scikit-learn, etc.).

## 9. Limitaciones

- Tamaño del dataset y variabilidad entre folds limitan la potencia estadística. - Preprocesado simple (imputación por 0) puede afectar a modelos sensibles a escalas; se recomienda normalizar/estandarizar y evaluar pipelines con GridSearchCV para tunear hiperparámetros.

## 10. Referencias

- Hastie, T., Tibshirani, R., Friedman, J. (2009). The Elements of Statistical Learning. - scikit-learn documentation: <https://scikit-learn.org>